EBU5204 Operating Systems 2023/24

Lab 3: Introduction to Python Scripts and Creating Processes using `fork()`

Aims

The aims of this lab are:

- to write simple Python scripts and to learn to process command line arguments
- to run and examine scripts that create new processes using the `os.fork()` function and to understand their behaviour.

You may find looking at the following resources to be useful. Note that Python 3.10 is used in the links below.

1. The official Python documentation: https://docs.python.org/3.10/
2. Introduction to Programming in Python - an on-line textbook: https://introcs.cs.princeton.edu/python/home/
3. Look at the functions available at the **`os` module**

   https://docs.python.org/3.10/library/os.html

4. MIT Lecture slides and code. MIT have a policy of making some of their material for past modules available to the public. https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/lecture-slides-code/

You should be able to open a Linux terminal and run a Python script. Here we are using python3 which invokes the python interpreter.

Consider the following code:

```
print("Hello World")
```

Assuming that the code is in a file 'hello.py', you can run the code by entering:

```
python3 hello.py
```

Alternatively, you can also run a Python script like an executable (assuming you have made it executable by giving it the correct permissions (see **Lab1**)) if you add the following as the **first line** of the script:

```
#!/usr/bin/env python3
```

`env` is an actual Linux program and here we assume it is in the `/usr/bin/` directory. In your case it may be somewhere else – check by typing: `which env` at a terminal. The path you get is the location of the env binary (executable) in your system.

Once you do this and after making sure you have made the file executable by using knowledge from **Lab 1** (e.g., `chmod 744 hello.py`) you can then execute it by typing: `./hello.py` without calling the Python interpreter. Note that "./" is pointing to the current directory. Depending on whether the current directory is in your `PATH` or not, leaving this out may lead to failure to find and execute the script.

PART A – Simple Python Scripts

(You can download the sample code from the scripting folder on QMplus)

1 Find Large Files – Method 1

You modify a script to find the "large" files in a directory. For simplicity, you can take an arbitrary fixed threshold for 'large' (such as 10,000 bytes, but you can decide on the threshold to make sure you have such files in your directory) and assume the files are in the current directory from where you run the script. Check of course that you have some "large" files in the directory where you run the script from, if not create or copy some.

The following script invokes the 'ls -l' command and creates a pipe from which the output can be read.

```
import os
# open a pipe to the output of the ls command
p = os.popen("ls -l")
# discard the first summary line of ls if it causes problems
p.readline()

for line in p:
      print(line.rstrip())
      # split each line of ls output into fields
      fields = line.split()
      # print each field
      i=0
      while i < len(fields) :
            print("Field", i, "is", fields[i])
            i +=1
```

Download the file **largeFiles1.py** from QMPlus **(don't copy & paste from the pdf)**. Modify this script to solve the "large" file problem:

- Your script should **only** print the name and size in bytes of every file with size over the threshold you set.
- Your script should check if there are no files found over the size threshold and output a message stating this.

2 Finding Large Files – Method 2

The script from the previous section depends on the exact format of the output of the 'ls –l' command. If this changes (e.g. between variants of Unix), the script will not be portable. It is preferable to use a Python function to determine the filesize, such as those defined in: https://docs.python.org/3.10/library/os.path.html

The following script shows how to get the file name and size without using 'ls':

```
import os
import glob
# the glob function returns a list of file names matching the pattern in the current directory. Pattern
* refers to all files in the current directory.
for filename in glob.glob('*'):
    print("File name", filename, "has size", os.path.getsize(filename),
    "bytes")
    # os.path.getsize returns the size in bytes
```

Download the file **largeFiles2.py (don't copy & paste from the pdf).** As before, read, copy and modify this script to solve the large file problem to output **only** the name and size of "large" files. Check if there are no large files found and output a message stating this.

3 Processing command line arguments

It would be better to specify the size as a command-line argument. For example, if you want to check if you have any files larger than 5,000 bytes in your current directory you should be able to pass the size as a command-line argument to your script, instead of hardcoding the size in your code, e.g.:

`python3 largeFiles.py 5000` or `./largeFiles.py 5000`

Enhance the script (based on either Method 1 or Method 2) to do this. Check if there are no large files found and output a message stating this. Perform basic checks for the argument passed (that there is exactly one argument passed, and that it is numeric). Output appropriate error messages if there are no (or more than one) arguments passed, or if the argument passed is not numeric.

See: https://docs.python.org/3.10/library/sys.html for the `sys.argv` function that helps you parse command line arguments.

PART B – Process creation using `fork`

This part complements the lecture material in Block 1 for process creation using the POSIX fork system call. **Note** that its implementation in the form of the `os.fork()` has one slight difference vs. the usual POSIX behaviour in terms of the return values: **If fork fails to create a new process** (e.g. maybe the system is out of resources and it can't create a new process) the standard **POSIX fork will return -1**. The Python implementation in **os.fork() just throws an OSError exception**.

The aim of this section is for you to run the code given, understand it, modify it as you need to understand its behaviour and, ultimately, understand why the output is as is. **Do not copy & paste the code from this pdf, download the source files from QMPlus.**

Use the resources at the top of the lab sheet for getting more information on the various functions you will see, the majority of which are in the `os` module.

1. Simple fork for creating a new process (script fork-simple.py)

```
import os

try :
      new_pid = os.fork()
      # fork creates a copy of the current process
      # the process id of the child process is returned if in the parent process
      # 0 is returned if in the child process
      # OSError exception is thrown if fork fails (note in POSIX this would return -1)
      #  both processes continue to this point, but when os.fork() returns,
      #  we have to find out if we are in the parent process or
      #  in the child process, both run concurrently!!!

      if (new_pid == 0) :
            print("In child and child pid is", os.getpid())
            print("My parent pid is", os.getppid())
            exit()
      else :
            print("Parent: pid", os.getpid(), "and child pid", new_pid)
except OSError:
        exit("Could not create child process")
```

2. Simple fork with wait to collect exit status from the child process - (script fork-wait.py)

Note that in the previous example it is very likely that the parent will finish first, and exit (i.e. the parent process will be then "dead", with no pid still assigned to it). The child will then proceed but e.g. when we ask it to print its parent id in the previous sample it will not have the correct id to print (because the parent will be dead).

To avoid this, it is good programming practice for the parent to **wait** for its child processes to finish and to **collect their exit codes**. The following sample code shows this. Run it, look up the functions you see and make sure you understand what goes on. Look at the different wait functions at the os module in Python.

```
import os

try :
      new_pid = os.fork()
      # fork creates a copy of the current process
      # the process id of the child process is returned if in the parent process
      #0 is returned if in the child process
      #OSError exception is thrown if fork fails (note in POSIX this would return -1)
      #  both processes continue to this point, but when os.fork() returns,
      #  we have to find out if we are in the parent process or
      #  in the child process, both run concurrently!!!

      if (new_pid == 0) :
            print("In child and child pid is", os.getpid())
            print("My parent pid is", os.getppid())
            exit()
      else :
            print("Parent: pid", os.getpid(), "and child pid", new_pid)

            # wait for the child process to exit
             # the 'status' variable is updated with the exit status
```

```
            # If we pass 0 to os.waitpid, the request is for the status of
            # any child in the process group of the current process.
            # If we pass -1, the request pertains to any child of the current process.
            # An alternative function to use is os.wait()
            _, status = os.waitpid(0, 0)

            # this function gets the exit code from the status
            exitCode = os.WEXITSTATUS(status)
            print("Child exit code is", exitCode)
except OSError:
       exit("Could not create child process")
```

3. Process races with fork - (script fork-races.py)

Look at the following script, run it and reason about its behaviour. Do small modifications as necessary to figure out what goes on, for example use/disable the sleep functions in one or both processes and notice any differences in the output, increase the range in the for loops to many thousand iterations, etc.

```
import os
import time

new_pid = os.fork()

if new_pid == 0:
    # We are in the child process, fork returned 0
    print(os.getpid(), "(child) just was created by", os.getppid())

     for i in range(0, -100, -1):
         print("Child:", i)
         # time.sleep(1)
         # seconds to sleep for
    exit()
else:
    # We are in the parent process, fork returned new_pid > 0, which is child pid
print(os.getpid(), "(parent) just created", new_pid)

     for i in range(100):
         print("Parent:", i)
         # time.sleep(1)
         # seconds to sleep for
```